

# Moving the Backend to Hugging Face Spaces

Why the backend could not stay where it was, what we considered, how we moved it, and what comes next.

Date: 29 September 2026 · Status: backend live; web app still needs to be pointed at it (section 8)

## 1. Summary

The Understand Kamba web app deploys to Vercel, but its Python backend (translation, speech recognition and voice) cannot run there: it needs about 5.5 GB of dependencies, about 7 GB of models and about 4.5 GB of memory, while Vercel functions are limited to 500 MB. Until the backend had a public home, the deployed site could not translate anything.

We compared paid hosts, free tiers and running it from a home PC, and moved the backend to a free Hugging Face Space on ZeroGPU hardware. The existing backend code runs there unchanged, wrapped in a small entry point. Getting it running took three fixes, each caused by how ZeroGPU and Gradio start apps. It is now live at <https://lawmaluki-understand-kamba-api.hf.space> and answers every endpoint in 1 to 2.5 seconds.

The last step is to set `NEXT_PUBLIC_API_BASE_URL` in Vercel and redeploy.

## 2. Where we started

The project has two parts:

- **Web app** (web/): a Next.js site. It runs in the visitor's browser and calls the backend over HTTP.
- **Backend** (app/): a FastAPI server running three models on the CPU: Meta NLLB-200 for translation, a w2v-BERT Kamba model for speech-to-text, and Meta's MMS Swahili voice for read-aloud.

Until now both ran only on a development PC, with the web app calling `http://localhost:8000`.

## 3. Why we had to move

### 3.1 Vercel could not build the project

The first Vercel deployment failed:

```
Error: Total bundle size (5513.37 MB) exceeds the maximum function size (500 MB).
```

Vercel saw `requirements.txt` at the top of the repository and tried to package the Python backend as a serverless function. PyTorch and transformers alone are about ten times Vercel's limit. Setting Vercel's Root Directory to web fixed the build: Vercel now deploys only the website, which is small.

### 3.2 The deployed site had nothing to talk to

With the website deployed, translation still failed with "Couldn't reach the translation server at http://localhost:8000". The browser console showed why:

```
Access to fetch at 'http://localhost:8000/translate' from origin
'https://understand-kamba.vercel.app' has been blocked by CORS policy:
Permission was denied for this request to access the `loopback` address space.
```

The deployed site had no backend address configured, so it fell back to localhost. Browsers block public websites from calling localhost, and even without that block, localhost means each visitor's own computer. The backend needed a public home.

### 3.3 What that home must provide

We measured the backend with all three models loaded:

| Loaded                           | Memory |
|----------------------------------|--------|
| Python, PyTorch and transformers | 0.3 GB |
| + translation (NLLB)             | 2.5 GB |
| + voice                          | 2.7 GB |
| + speech-to-text                 | 4.5 GB |

So the host needs:

- at least 4.5 GB of memory, ideally 8 GB for headroom;
- about 10 GB of disk for dependencies and models;
- a long-running server process (serverless platforms reload models on every cold start);
- an HTTPS address, because an HTTPS page cannot call an HTTP API.

## 4. Options we considered

| Option                                  | Cost                     | Verdict                                                                                     |
|-----------------------------------------|--------------------------|---------------------------------------------------------------------------------------------|
| Vercel                                  | —                        | Impossible: 500 MB function limit.                                                          |
| Render Free, Starter or Standard        | Free to about \$25/month | Too little memory (512 MB to 2 GB).                                                         |
| Render Pro, 4 GB                        | About \$85/month         | Just under the 4.5 GB needed. Could work if the models were quantized to use less memory.   |
| Render Pro Plus, 8 GB                   | About \$175/month        | Works, but costly for an experiment.                                                        |
| Railway, Koyeb or Fly.io free tiers     | Free                     | Too little memory.                                                                          |
| Google Colab                            | Free                     | Sessions end after a few hours.                                                             |
| Oracle Cloud Always Free                | Free                     | Enough memory, but needs a credit card and manual server setup.                             |
| Development PC plus a Cloudflare Tunnel | Free                     | Fastest to set up, but only works while the PC is on, and the address changes each restart. |

| Option                                      | Cost        | Verdict                                                                  |
|---------------------------------------------|-------------|--------------------------------------------------------------------------|
| Hugging Face Space, Docker                  | Paid        | We planned this, but the Docker option was marked Paid for this account. |
| Hugging Face Space, CPU Basic               | Paid        | Switching to it returned "402 Payment Required".                         |
| <b>Hugging Face Space, ZeroGPU (Gradio)</b> | <b>Free</b> | <b>Chosen.</b>                                                           |

Render prices are as we understood them at the time; check [render.com/pricing](https://render.com/pricing) before relying on them.

Two of our assumptions turned out to be out of date: Hugging Face now charges for Docker Spaces and for CPU Basic hardware on this account. ZeroGPU with the Gradio SDK was the only free option left, and it came with its own rules (section 6).

## 5. The decision

We chose a free Hugging Face Space on ZeroGPU hardware because:

- **It costs nothing**, which suits a project still in its learning phase.
- **It has enough memory**. All three models load and run comfortably.
- **It is fast**. Models download from Hugging Face's own network at hundreds of MB per second, so a restart takes about a minute rather than many.
- **It gives a stable HTTPS address** that does not depend on anyone's PC being on.
- **The backend code did not need to change**. Everything Space-specific lives in a separate space/ folder.

We accepted these trade-offs:

- The Space sleeps after a couple of days without visitors; the next visit is slow while it wakes.
- Storage is temporary, so stats and ratings reset when the Space restarts.
- ZeroGPU has rules that ordinary servers do not (section 6), and they could change.

One deliberate design choice: **the models run on the CPU, not the GPU**. ZeroGPU lends a GPU for a few seconds at a time, but charges each visitor's daily quota: 2 minutes a day for anonymous visitors and 5 minutes for logged-in free accounts. Our web app calls the API anonymously, so heavy use would soon hit "quota exceeded" errors. On the CPU there is no quota, and the speed turned out to be good.

## 6. How we did it

### 6.1 Split the deployment

- **Website on Vercel**, with Root Directory set to web.
- **Backend on the Space**.

## 6.2 Create the Space

Created at [huggingface.co/spaces/lawmaluki/understand-kamba-api](https://huggingface.co/spaces/lawmaluki/understand-kamba-api) with the Gradio SDK, ZeroGPU hardware, public visibility and a CC-BY-NC 4.0 licence (matching the non-commercial licences of the NLLB and MMS models). Public is required: the website calls the API from visitors' browsers, and a private Space would reject those calls.

## 6.3 Add a Space entry point, leaving the backend unchanged

The repository gained a `space/` folder:

| File                                | Purpose                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>space/server.py</code>        | Starts Gradio, adds a small test page at <code>/</code> , and attaches the backend's existing API routes ( <code>/translate</code> , <code>/transcribe</code> , <code>/synthesize</code> , <code>/stats</code> , <code>/feedback</code> , <code>/health</code> , <code>/docs</code> ) to Gradio's server. It also loads the models in the background at startup, so the first visitor does not wait for downloads. |
| <code>space/README.md</code>        | The Space's configuration header (SDK, Gradio version 6.28.0, Python 3.12, entry file, licence) and a short description.                                                                                                                                                                                                                                                                                           |
| <code>space/requirements.txt</code> | Backend dependencies, with PyTorch pinned to 2.13.0, the newest version ZeroGPU supports.                                                                                                                                                                                                                                                                                                                          |

A deploy script, `scripts/deploy_space.py`, copies `space/` and `app/` into a temporary folder and uploads only that. The web app, `docs`, `.env` and local data never reach Hugging Face.

## 6.4 Three problems and their fixes

Before each upload we ran the entry point locally with the same Gradio version. The Space still needed three attempts, because ZeroGPU and Gradio behave differently on Hugging Face than on a PC.

**Attempt 1: the Space crashed at startup.** The logs showed two errors:

```
No @spaces.GPU function detected during startup
[Errno 98] error while attempting to bind on address ('0.0.0.0', 7860): address already in use
```

Reading the source of Hugging Face's spaces package showed that ZeroGPU hooks into Gradio's `demo.launch()`. At that moment it reports the app's GPU functions to the platform, and the platform refuses to start an app that declares none. Our first entry point started its own web server and never called `demo.launch()`.

Fix: import spaces before anything that loads PyTorch, declare one small function marked `@spaces.GPU` that is never called, and start the app through `demo.launch()`.

**Attempt 2: the Space ran, but the API returned "405 Method Not Allowed".** `/health` and `/stats` worked, but every POST, such as `/translate`, failed, and GET `/translate` returned a web page instead of our API.

On Spaces, recent Gradio versions run a Node.js server-side rendering (SSR) server on port 7860 in front of Python. It forwards only Gradio's own paths, so our routes never reached Python. This SSR server was also what held port 7860 in attempt 1.

Fix: launch with `ssr_mode=False`, so Gradio's Python server serves port 7860 directly.

**Attempt 3: everything worked.**

Cross-origin access needed no changes. Gradio has its own cross-origin (CORS) handling, which allows requests from any website when it is served from a public address such as `*.hf.space`. We confirmed this in

Gradio's source before relying on it, then tested it live.

## 6.5 Verification

We called the live API exactly as the website does, with the Vercel site as the origin:

| Check                                                   | Result                                                         | Time  |
|---------------------------------------------------------|----------------------------------------------------------------|-------|
| Cross-origin preflight from understand-kamba.vercel.app | Allowed                                                        | —     |
| GET /health, GET /stats                                 | 200                                                            | 0.3 s |
| English to Kikamba                                      | "Nĩngũkwĩsũva ũete kĩw'ũ kuma ũsĩnĩ ũsu ũtanamba kũvika wĩoo." | 1.6 s |
| Kikamba to English, round trip                          | "Please bring water from the river before evening."            | 1.1 s |
| Read aloud                                              | WAV audio returned                                             | 2.3 s |
| Speech-to-text of a browser-style .webm recording       | "ningukuisuvo uete kiu kuma usini usukuta na mbakuvika wioo"   | 1.4 s |

In these tests the Space's CPU was faster than the development PC, and no GPU quota was used.

## 7. How it fits together now

| Piece                  | Where it runs                                        | Address                                                                                                       |
|------------------------|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Website                | Vercel                                               | <a href="https://understand-kamba.vercel.app">https://understand-kamba.vercel.app</a>                         |
| API and models         | Hugging Face Space (ZeroGPU hardware, CPU inference) | <a href="https://lawmaluki-understand-kamba-api.hf.space">https://lawmaluki-understand-kamba-api.hf.space</a> |
| API docs and test page | Same Space                                           | /docs and /                                                                                                   |
| Source code            | GitHub                                               | <a href="https://github.com/lawmaluki/understand-kamba">github.com/lawmaluki/understand-kamba</a>             |

A visitor's browser loads the website from Vercel, then calls the Space directly for every translation, transcription, voice clip, rating and stats lookup. Vercel never talks to the Space itself.

To update the backend after changing app/ or space/:

```
python scripts/deploy_space.py
```

The Space rebuilds and restarts in about two minutes. Build and run logs are at [huggingface.co/spaces/lawmaluki/understand-kamba-api](https://huggingface.co/spaces/lawmaluki/understand-kamba-api) under Logs.

## 8. Immediately outstanding

- 1 **Point the website at the Space.** In Vercel, go to Settings, then Environment Variables, and add `NEXT_PUBLIC_API_BASE_URL` with the value `https://lawmaluki-understand-kamba-api.hf.space` (no trailing slash). Then redeploy: the address is built into the site at build time.
- 2 **Replace the Hugging Face token.** The write token was pasted into a chat during setup. Delete it at `huggingface.co/settings/tokens`, create a new one, and log in again with hf auth login. The deploy script uses whatever token that login saves.
- 3 **Commit the Space files.** `space/` and `scripts/deploy_space.py` exist locally but are not yet in git.

## 9. Risks and limitations

| Risk                                                    | Impact                                                                                                                                                                                              | Mitigation                                                                                                                          |
|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| The Space sleeps after a couple of days without traffic | The first visit afterwards waits while it wakes and loads models                                                                                                                                    | Show a "waking up, please wait" message in the web app instead of an error (section 10).                                            |
| Temporary storage                                       | Stats and ratings reset on every restart                                                                                                                                                            | Move them to persistent storage (section 10).                                                                                       |
| Open API                                                | Anyone who finds the address can use it. There is no login or rate limit, and Gradio's CORS handling accepts every website, so the API cannot be limited to the Vercel site                         | Add rate limiting; watch usage.                                                                                                     |
| Relies on ZeroGPU and Gradio internals                  | The unused GPU function, <code>ssr_mode=False</code> and attaching routes to Gradio's server work today, but a Gradio or ZeroGPU update could break them                                            | Gradio (6.28.0) and PyTorch (2.13.0) are pinned. Test locally before changing versions, and check the Space logs after each deploy. |
| Hugging Face policy changes                             | Docker and CPU Basic became paid during this work; ZeroGPU could change too. Free ZeroGPU hosting requires a verified email and an account older than 30 days, and allows at most 2 ZeroGPU Spaces. | The Dockerfile from commit <code>d890b0b</code> keeps a paid-host option ready (see section 10).                                    |
| Non-commercial licences                                 | NLLB-200 and the MMS voice are CC-BY-NC 4.0                                                                                                                                                         | Fine for learning and research; replace these models before any commercial use.                                                     |

## 10. What next

### Short term

- Finish the three items in section 8.
- **Handle cold starts in the web app.** When a request times out or the Space is waking, show "The translation service is waking up; this can take a minute" and retry, instead of the current error.
- **Keep stats and ratings across restarts.** Options: a Hugging Face storage bucket attached to the Space (the create form offers one; check its current pricing), or a small free external database (for example a free Postgres tier) written to by `app/stats.py`.
- **Add basic rate limiting** per IP address to protect the free Space from abuse.

## Medium term

- **Measure translation quality.** Build a test set of 30 to 50 sentences with Kikamba translations checked by a speaker, and score the current model against it. The ratings log (`data/feedback.jsonl`) is a good source of sentences that went wrong. Once ratings persist, review them regularly.
- **Try a larger translation model.** `facebook/nllb-200-distilled-1.3B` usually translates better. The Space has the memory for it; measure speed and quality against the test set before switching.
- **Automate deploys.** A GitHub Action could run `scripts/deploy_space.py` whenever `app/` or `space/` changes on main, so the backend never falls out of step with the code.

## When usage grows

- **Move to paid, always-on hosting.** The Dockerfile and configurable CORS on the `feature/mobile-and-deploy` branch (commit `d890b0b`) run the same backend on Render, Railway, Fly.io or a VPS with about 8 GB of memory. Quantizing the translation model could let it fit a cheaper 4 GB plan.
- **Consider the GPU for heavier models** only once visitors can log in with Hugging Face, since ZeroGPU quota is charged per visitor.
- **Replace the non-commercial models** before any commercial use.